

به نام خدا



درس سیستم عامل
نیم سال اول ۰۵-۰۴
استاد: دکتر اسدی

دانشکده مهندسی کامپیوتر

تمرین سری اول

- پرسش‌های خود را در سامانه CW و تالار مربوط به تمرین مطرح نمایید.
- پاسخ سوالات را تایپ نمایید.
- اسکرین‌شات‌ها، عکس‌ها، فایل‌های مربوط به سوال عملی، گزارش تمرینات عملی و PDF قسمت تئوری را در پوشه با نام به فرمت HWNUM_StudentID1_StudentID2 ذخیره نمایید. سپس آن را zip نمایید و در صفحه درس بارگذاری نمایید. به عنوان مثال یک فایل بارگذاری شده قابل قبول باید دارای فرمت HW1_400123456_403123456.zip باشد.
- هر دانشجو می‌تواند حداکثر سه تمرین را با دو روز تأخیر بدون کاهش نمره ارسال نماید.
- تمرینات عملی و تئوری به صورت گروه‌های دو نفر تحویل داده شود.
- هر دو عضو گروه موظف هستند تمرینات خود را بارگذاری کنند.
- عواقب عدم تطابق بین پاسخ دو عضو گروه برعهده خودشان است.
- تحویل تمرین به صورت انگلیسی مجاز نیست. در صورت تحویل تمرین به صورت انگلیسی (حتی بخشی از تمرین) نمره تمرین موردنظر صفر در نظر گرفته می‌شود.
- در صورت مشاهده تقلب برای بار اول نمره هر دو طرف صفر می‌شود. در صورت تکرار نمره کل تمرینات صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- توجه شود که پروژه نهایی درس در گروه‌های چهار نفر تحویل گرفته می‌شود.
- سوالات با عنوان اختیاری نمره‌ای ندارند اما جواب دادن به آن‌ها کمک به‌سزایی در یادگیری درس می‌کند.
- هیچ تمرینی خارج از CW تصحیح نخواهد شد. لذا توصیه می‌شود حداقل یک الی دو ساعت قبل از زمان پایان تمرین برای بارگذاری تمرین اقدام شود.

تمارین تئوری

۱. کد زیر را در نظر بگیرید:

```

1  #include <stdio.h>
2  #include <sys/resource.h>
3
4  int main() {
5      struct rlimit lim;
6
7      getrlimit(RLIMIT_STACK, &lim);
8      printf("stack size: %ld\n", lim.rlim_cur);
9
10     getrlimit(RLIMIT_NPROC, &lim);
11     printf("process limit: %ld\n", lim.rlim_cur);
12
13     getrlimit(RLIMIT_NOFILE, &lim);
14     printf("max file descriptors: %ld\n", lim.rlim_cur);
15     return 0;
16 }
```

آ) توضیح دهید این کد چه کاری انجام می‌دهد. برای هر کدام از `RLIMIT_NOFILE` و `RLIMIT_NPROC` و `RLIMIT_STACK` بیان کنید این محدودیت‌ها چه مفهومی دارند و واحد آن‌ها چیست.

ب) ساختار `struct rlimit` را توضیح دهید (شامل `rlim_max` و `rlim_cur`) و فرق بین `hard` و `soft limit` را بنویسید.

ج) توضیح دهید که چگونه محدودیت‌ها در سطح سیستم تغییر می‌کنند (مانند استفاده از دستور `ulimit`، فایل `/etc/security/limits.conf` در لینوکس، یا تنظیمات `systemd`) و چه سطح دسترسی برای تغییر آن‌ها لازم است.

د) کاربردهای تنظیم این محدودیت‌ها و خطرات تغییر نادرست آن‌ها (مانند وقوع `fork-bomb`، مصرف بیش از حد فایل‌ها، یا خطاهای مربوط به سرریز پشته^۱) را بررسی کنید.

۲. به سؤالات زیر پاسخ دهید:

آ) یک پردازشی کاربر^۲ چگونه می‌تواند باعث شود عملیات با سطح دسترسی ویژه^۳ انجام شوند؟

ب) سه روش نام ببرید که در آن‌ها پردازنده می‌تواند از حالت کاربر^۴ به حالت هسته^۵ منتقل شود. آیا کاربر بعد از این انتقال می‌تواند هر کدی که بخواهد اجرا کند؟ توضیح دهید.

ج) برنامه‌هایی که در یک سیستم چندبرنامه‌ای^۶ در حال سرویس‌دهی هستند با $P_1, P_2, \dots, P_m$ نشان داده می‌شوند، که در آن m درجه‌ی چندبرنامه‌ای بودن سیستم است. تمامی برنامه‌ها دارای ماهیتی چرخه‌ای هستند، به طوری که هر چرخه شامل یک دوره‌ی فعالیت پردازشی و یک دوره‌ی فعالیت ورودی/خروجی است. فرض کنید b_{io}^i و b_{cpu}^i به ترتیب نشان دهنده‌ی زمان‌های اجرای پردازشی^۷ و اجرای ورودی/خروجی^۸ برنامه‌ی P_i باشند. در مورد درستی یا

¹Stack Overflow

²User Process

³Privileged Operations

⁴User Mode

⁵Kernel Mode

⁶Multi-Programming System

⁷CPU burst

⁸I/O burst

نادرستی گزاره‌ی زیر توضیح دهید: (P_h برنامه‌ی با بالاترین اولویت است).

$$b_{io}^h > \sum_{j \neq h} b_{cpu}^j$$

پردازنده مرکزی بیکار می‌شود اگر:

۳. در رابطه با وقفه‌ها^۹ و استثناها^{۱۰} به سؤالات زیر پاسخ دهید.

- (آ) راجع به طوفان وقفه^{۱۱} تحقیق کنید و یکی از سازوکارهای سیستم عامل برای مدیریت و کاهش آن را توضیح دهید.
- (ب) وقفه‌های سخت‌افزاری و نرم‌افزاری را با یکدیگر مقایسه کنید و برخی از تفاوت‌های آن‌ها را بیان کنید.
- (ج) نقش جدول بردار وقفه^{۱۲} در رسیدگی به وقفه چیست و چرا برای عملکرد قابل اعتماد سیستم عامل لازم است؟
- (د) در ابتدا توضیح دهید پوشاندن^{۱۳} یک وقفه به چه معناست و چرا در سیستم عامل به آن نیاز است؛ در ادامه تحقیق کنید که برای چه وقفه‌ای امکان پوشش وجود ندارد؟
- (ه) یک پردازنده عددی را بر صفر تقسیم می‌کند. مرحله‌ی را که سیستم عامل از لحظه‌ی بروز استثنا تا بازگرداندن کنترل به یک حالت ایمن انجام می‌دهد را توضیح دهید.
- (و) چرا برخی از استثناها در داخل هسته رسیدگی می‌شوند و پردازنده پس از آن ادامه می‌یابد، در حالی که برخی دیگر منجر به توقف اجرای پردازنده می‌شوند؟

۴. به سؤالات زیر پاسخ دهید:

(آ) شرایط زیر را در یک رایانه شخصی تک کاربره در نظر بگیرید:

- ۱) یک موس که با رابط کاربری گرافیکی مورد استفاده قرار می‌گیرد،
- ۲) یک دیسک درایو که شامل فایل‌های کاربر است،
- ۳) یک کارت گرافیک با اتصال مستقیم به گذرگاه حافظه، که از طریق Memory-Mapped I/O قابل دسترسی است.

برای هر یک از این سناریوهای ورودی/خروجی، مشخص کنید که آیا از Polled I/O یا Interrupt-Driven I/O استفاده می‌کنید؟

(ب) کدام یک از دستورات زیر باید از نوع «ویژه» یا «سطح بالا»^{۱۴} باشند؟

- ۱) تنظیم مقدار زمان‌بند^{۱۵}
- ۲) خواندن ساعت سیستم
- ۳) پاک کردن حافظه
- ۴) اجرای دستور تله^{۱۶}
- ۵) دسترسی به دستگاه‌های ورودی/خروجی

(ج) هدف از وقفه‌ها چیست؟ تفاوت بین یک تله و یک وقفه در چیست؟ آیا تله می‌تواند عمده‌اً توسط برنامه کاربر ایجاد شود؟ اگر بله، با چه هدفی؟

⁹Interrupts

¹⁰Exceptions

¹¹Interrupt Storm

¹²Interrupt Vector Table

¹³Interrupt Masking

¹⁴Privileged

¹⁵Timer

¹⁶Trap

(د) سیستم عامل یک فراخوانی سیستمی برای درخواست تخصیص حافظه فراهم می‌کند. یک برنامه‌نویس با تجربه توصیه‌ای ارائه داده است: «اگر برنامه شما شامل درخواست‌های متعدد برای تخصیص حافظه است، می‌توانید با ادغام همه این درخواست‌ها در یک فراخوانی سیستمی، اجرای برنامه را سریع‌تر کنید.» دلیل این توصیه را توضیح دهید.

۵. یک سیستم کامپیوتری از دسترس مستقیم به حافظه (DMA)^{۱۷} برای انتقال داده پرسرعت بین کنترل‌کننده دیسک و حافظه اصلی استفاده می‌کند. فرض کنید پردازنده مرکزی باید ۱ مگابایت داده از دیسک به حافظه منتقل کند.

(آ) مرحله به مرحله توصیف کنید که چگونه CPU، کنترل‌کننده DMA و حافظه با هم در این انتقال تعامل می‌کنند. به‌طور دقیق توضیح دهید چه چیزی توسط CPU تنظیم می‌شود، کنترل‌کننده DMA چه کاری انجام می‌دهد و چگونه CPU مطلع می‌شود.

(ب) فرض کنید گذرگاه حافظه توان پردازش ۱۰۰ مگابایت بر ثانیه را دارد و CPU می‌تواند هم‌زمان دستوراتی اجرا کند که فشاری بر حافظه وارد نمی‌کنند. زمان تخمینی انتقال DMA چقدر است (بدون احتساب سربار وقفه)؟ اگر فرکانس CPU برابر ۱ گیگاهرتز باشد، چه تعداد چرخه‌ی CPU دزدیده^{۱۸} می‌شود؟

(ج) در طول انتقال DMA، چه نوع تداخل یا رقابتی ممکن است بین کنترل‌کننده DMA و CPU رخ دهد؟ چگونه می‌توان این تداخل را کاهش داد یا مدیریت کرد؟

(د) یک روش جایگزین را با استفاده از ورودی/خروجی برنامه‌ریزی شده^{۱۹} برای همان انتقال ۱ مگابایت مقایسه کنید. کدام روش مؤثرتر است و در چه شرایطی ورودی/خروجی برنامه‌ریزی شده قابل قبول است؟

۶. فرض کنید کامپیوتری را روشن می‌کنید تا سیستم عامل آن بوت شود.

(آ) مراحل مختلف بوت شدن یک سیستم را از لحظه‌ی فشار دکمه‌ی روشن کردن تا زمانی که هسته کامل بارگذاری شده و آماده‌ی اجرای فرآیندهای کاربر است، به ترتیب توضیح دهید. نام مؤلفه‌ها و وظایف هر مرحله (مثلاً ثابت‌افزار^{۲۰}، BIOS، Boot Block، Bootstrap Loader، یا UEFI) را مشخص کنید.

(ب) توضیح دهید که چرا نیاز به برنامه‌ای به نام Boot Loader وجود دارد. چه کارهایی انجام می‌دهد که ثابت‌افزار به تنهایی انجام نمی‌دهد؟

(ج) اگر بخواهیم بوت را در سیستم‌های جدید با UEFI را با سیستم‌های قدیمی با BIOS و MBR مقایسه نماییم، تفاوت‌های کلیدی چیست؟ مزایا و معایب هر کدام را بررسی کنید.

(د) فرض کنید Boot Loader نام هسته را اشتباه دارد یا هسته آسیب دیده است. چه اتفاقی می‌افتد؟ چگونه سیستم می‌تواند به وضعیت تعمیر^{۲۱} برود؟

¹⁷Direct Memory Access

¹⁸Stolen

¹⁹Programmed I/O

²⁰Firmware

²¹Recovery

تمارین عملی

۱. یکی از سوالات تمرینات عملی این درس مبتنی بر سیستم عامل xv6 ساخته شده توسط دانشگاه MIT است که یک سیستم عامل برای اهداف آموزشی است. لینک مستندات و فیلم های آموزشی مربوط به این سیستم عامل در صفحه ی cw درس قرار داده شده است.

آ- با استفاده از فیلم آموزشی موجود در cw سیستم عامل xv6 را اجرا کنید و از مراحل آن اسکرین شات تهیه کنید. پیشنهاد ما این است که نسخه ی ۱۳ توزیع Debian و یا Ubuntu 24 استفاده کنید. در صورتی که از سیستم عامل لینوکس استفاده نمی کنید می توانید به کمک لینک زیر یک ماشین مجازی Debian و یا Ubuntu بالا بیاورید:

<https://www.makeuseof.com/how-to-install-debian-on-virtualbox/>

<https://www.geeksforgeeks.org/linux-unix/how-to-install-ubuntu-on-virtualbox/>

لطفاً توجه کنید که حداقل نسخه مورد نیاز Qemu برای اجرای XV6، نسخه ۶.۲ است. اگر قصد استفاده از توزیع های دیگر لینوکس را دارید، حتماً به این نکته توجه کنید.

ب- در این بخش قصد داریم تا یک برنامه کاربر به سیستم عامل اضافه کنیم تا با اجرای آن عبارت **Hello World!** در خروجی ترمینال چاپ شود.

بدین منظور نیاز است تا تغییراتی را در بخش کاربر^{۲۲} سیستم عامل اعمال کنید و همچنین این برنامه جدید را به Makefile نیز اضافه کنید.

در نهایت این برنامه کاربر را اجرا کنید و خروجی آن را به همراه فایل های اضافه شده و یا تغییر داده شده آپلود کنید.

ج- **اختیاری:** سیستم عامل های POSIX-compliant چه سیستم عامل هایی هستند؟

۲. ما در این سری تمرینات عملی قصد داریم ابزار داکر^{۲۳}، یا به طور کلی ابزارهای مدیریت کانتینر^{۲۴}، را از جنبه های کاربرد، معماری و پیاده سازی با تمرکز بر مفاهیم سیستم عامل بررسی کنیم.

هشدار: انجام این تمرین و سایر تمارین مربوط به داکر تنها در سیستم عامل لینوکس پذیرفته است و در غیر این صورت تمرین تصحیح نخواهد شد.

هدف: در این تمرین هدف آشنایی مقدماتی با نحوه کار با داکر و بررسی چند مفهوم در رابطه با آن است.

مقدمه: در ابتدا، سه مفهوم ایمج^{۲۵}، کانتینر و رجیستری^{۲۶} را به همراه داکر، به صورت ناقص (و حتی شاید غلط!) تعریف می کنیم.

داکر ابزاری برای مدیریت کانتینرها است؛ کانتینرها محیطی ایزوله برای اجرای برنامه ها بر اساس ایمج مخصوص به خودشان هستند؛ ایمج ها ساختاری کلی برای اجرای برنامه ها هستند؛ و رجیستری محلی است که ایمج ها در آن قرار گرفته اند (این تعاریف به مرور با انجام تمارین برای شما کامل خواهد شد و اشکالات موجود در آن ها اصلاح می شود).

گام های تمرین:

آ. داکر را نصب کنید. می توانید از این لینک استفاده کنید.

ب. ما در این تمرین از ایمج alpine استفاده می کنیم. alpine یک نسخه بسیار فشرده و سبک از لینوکس است. ابتدا لازم است که این ایمج را از رجیستری Docker Hub دریافت کنید. به این منظور به واسط دستور زیر ایمج alpine با 3.22.2 TAG را دریافت کنید.

```
1 sudo docker pull alpine:3.22.2
```

²²User level

²³docker

²⁴Container

²⁵Image

²⁶Registry

سوال: گاهی به علت تحریم و یا فیلترینگ دریافت ایمیج‌ها با اختلال مواجه می‌شود. چگونه می‌توان دستورات داکر را با پروکسی استفاده کرد؟
حال به واسطه دستور زیر می‌توانید لیست ایمیج‌های موجود بر روی ماشین خود را مشاهده کنید.

```
1 sudo docker images
```

ج. برای اجرای یک برنامه در بستر یک ایمیج لازم است یک کانتینر بسازید و به وسیله آن کانتینر را اجرا کنید. به این منظور دستور زیر را اجرا کنید تا یک برنامه ساده را داخل کانتینر اجرا کنید.

```
1 sudo docker run alpine:3.22.2 sleep 10000
```

در صورت که هیچ اتفاقی نیفتاد و دستور اجرا شده فریز شد، شما کانتینر را با موفقیت ساخته‌اید. به منظور اطمینان از ساخته شدن کانتینر در یک ترمینال جدید دستور زیر را اجرا کنید.

```
1 sudo docker ps
```

در صورتی که کانتینر در حال اجرا باشد، اطلاعاتی در رابطه با آن کانتینر را می‌توانید مشاهده کنید.
د. به ترمینالی که در آن کانتینر را اجرا کرده بودید بازگردید و به واسطه دستور Ctrl+C اجرا آن کانتینر را متوقف کنید و دوباره لیست کانتینرهای در حال اجرا را بررسی کنید.
سوال: آیا راهی وجود دارد که لیست کانتینرهای متوقف شده را مشاهده کرد؟
سوال: اگر در خروجی دستور sudo docker ps دیده باشید، یک فیلد به عنوان اسم کانتینر وجود دارد. چگونه می‌توان یک کانتینر با اسم دلخواه اجرا کرد؟
ه. یک کانتینر مشابه با بخش قبل به اسم iloveos بسازید.
و. برای اجرای یک دستور در داخل یک کانتینر در حال اجرا می‌توانید از دستور زیر استفاده کنید.

```
1 sudo docker exec -it iloveos sh
```

به این صورت شما یک پوسته داخل محیط کانتینر در حال اجرا خواهید داشت.
ز. با استفاده دستور ps می‌توان لیست پردازش‌های در حال اجرا در سیستم عامل را به همراه pid آنها بررسی کرد. دستور زیر را یک بار در پوسته سیستم عامل و یک بار در پوسته ساخته شده در کانتینر اجرا کنید و برای دستور sleep 10000 مقدار pid را در دو محیط مقایسه کنید (در این تمرین نیازی به تحلیل این نتایج نیست).

```
1 ps aux | grep "sleep 10000" | grep -Ev "grep|docker"
```

سوال: وجود بخش سوم در کامند بالا چه فایده‌ای دارد؟
سوال: اجرای دستورات داکر به علت نیازمندی به دسترسی‌های سطح سیستم همواره نیازمند به sudo در ابتدا است. آیا راهی وجود دارد که دستورات را بدون sudo بدون استفاده از کاربر root اجرا کرد؟

موارد لازم جهت تحویل: برای دریافت نمره کامل لازم است موارد زیر در گزارش شما قرار داشته باشد:

- پاسخ به بخش‌های مطرح شده به فرمت «سوال» به صورت خلاصه.
- توضیح خلاصه و عکس از گام‌های ب، ج، د و ز.

۳. هدف از این تکلیف، آشنایی عملی با فرآیند کامپایل لینوکس و افزودن مازول‌های سفارشی برای اهداف توسعه و دیباگ است. در این تمرین، به طور خلاصه باید کارهای زیر انجام دهید:

- کد منبع هسته لینوکس را دانلود و پیکربندی کنید.
- مازول‌های خاص مورد نیاز برای دیباگ را به پیکربندی هسته اضافه کنید.

- هسته را با پیکربندی جدید کامپایل کنید.
- یک ماژول ساده هسته (.ko) را بنویسید و کامپایل کنید.
- ماژول را در هسته کامپایل شده خود اضافه و حذف کنید.

پیش‌نیازها

- یک سیستم مجازی یا فیزیکی با یک توزیع لینوکس (ترجیحا دیان ۱۲)
- توجه کنید که : نام کاربری این سیستم عامل را برابر با نام خود و هم گروهی هایتان قرار دهید

```
osvm@lastname0-lastname1$
```

- حداقل ۱۵ گیگابایت فضای آزاد دیسک
- ابزارهای ضروری برای کامپایل (build-essential, libssl-dev, flex, bison, etc.)

گام‌ها

۱. دریافت کد منبع هسته

۱. از وبسایت <https://www.kernel.org>، کد منبع نسخه 6.1.156 را دریافت کنید.

۲. پیکربندی هسته

۱. فایل پیکربندی فعلی سیستم خود را کپی کنید تا یک فایل پیکربندی پایه داشته باشید.

```
cp /boot/config-\$(uname -r) .config
```

۲. منوی پیکربندی هسته را با دستور `make menuconfig` اجرا کنید.

۳. در منوی پیکربندی، ماژول‌های زیر را فعال کنید:

- Kernel hacking -> Kernel debugging
- Kernel hacking -> Memory Debugging (گزینه‌هایی مانند KASAN را فعال کنید)
- Enable loadable module support (در صورت غیرفعال بودن)

۴. نکته : با استفاده از دستور `make localmodconfig` می‌توانید با حذف درایورهای غیر ضروری از `build` سرعت کامپایل را افزایش دهید.

۳. کامپایل هسته

۱. با استفاده از دستورات زیر هسته و ماژول‌ها را کامپایل کنید. این مرحله ممکن است مدت زمان زیادی طول بکشد.

```
make -j$(nproc)
sudo make modules_install
sudo make install
```

۲. GRUB را بروزرسانی کنید و سیستم را `reset` کنید و هسته جدید را انتخاب نمایید.

۴. توسعه و تست یک ماژول ساده

۱. یک ماژول ساده هسته به نام `mymodule.c` ایجاد کنید که هنگام بارگذاری و حذف شدن، پیامی را در `/var/log/kern.log` چاپ کند.
۲. یک `Makefile` ساده برای کامپایل ماژول بنویسید.
۳. ماژول را با استفاده از `make` کامپایل کنید.
۴. ماژول را با دستور `sudo insmod mymodule.ko` بارگذاری کرده و با دستور `sudo rmmod mymodule` حذف کنید.
۵. خروجی لاگ را با دستور `tail | dmesg` یا بررسی فایل `/var/log/kern.log` مشاهده کنید.

خروجی‌های قابل تحویل

لطفاً فایل‌های زیر را در یک آرشیو فشرده (`.zip` یا `.tar.gz`) ارسال کنید:

۱. فایل پیکربندی نهایی هسته شما (`.config`).
۲. کد منبع ماژول ساده شما (`mymodule.c`).
۳. `Makefile` مربوط به ماژول.
۴. یک گزارش کوتاه (`report.txt` یا `report.pdf`) که شامل موارد زیر است:
 - نسخه هسته‌ای که کامپایل کردید.
 - لیستی از ماژول‌های اصلی که برای دیباگ فعال کردید.
 - اسکرین‌شات از پیام‌های موفقیت‌آمیز بارگذاری و حذف ماژول شما در لاگ.
 - هر گونه خطا یا مشکل غیرمنتظره که با آن مواجه شدید و نحوه رفع آن.